PROJECT OVERVIEW

Asisten — AI Chatbot

A web chatbot whose answers stream word by word and can be stopped at any time, backed by a Go server that can use Gemini, OpenAI, or Anthropic.

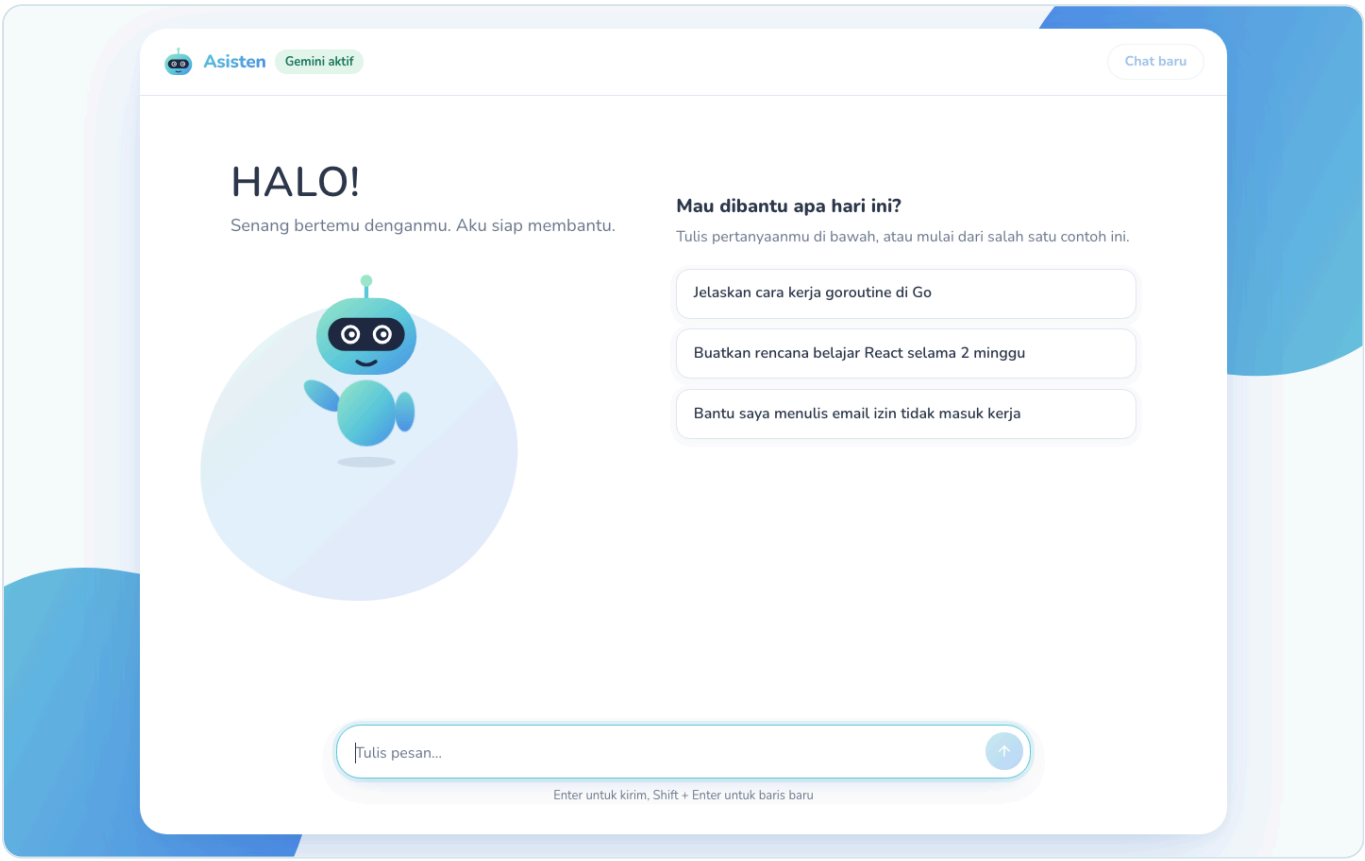
Raihan Evanza · Fullstack & Mobile Developer · portfolio-raihan-evanza.vercel.app

ROLE
Personal project · Go backend, React frontend, UI design

PLATFORMS
Web (desktop & mobile), runs locally

LINKS
Source code (GitHub): github.com/raihanevanza/chat-bot-go

Go 1.22 (standard library) · React 18 · Vite 5 · Server-Sent Events · Gemini API · OpenAI API · Anthropic API



OVERVIEW

A chatbot that waits for the full answer before showing anything feels slow, because a language model can take several seconds to respond. I built this project to learn how to show answers progressively, and to practise writing a backend in Go.

The result is a chat app with a Go backend that uses only the standard library and a React + Vite frontend. The backend relays answer chunks from the AI provider to the browser over Server-Sent Events, and the user can stop an answer midway. The provider is picked automatically from whichever API key is set (Gemini, OpenAI, or Anthropic).

With no API key at all, the backend runs in demo mode and still streams a sample answer, so anyone can clone the repo and try the full flow in a few minutes at no cost.

KEY FEATURES

Real-time streaming answers

Each text chunk from the model is sent to the browser as an SSE event and appended to the chat bubble, so the first words appear without waiting for the full answer.

Stop an answer midway

While an answer is streaming, the send button turns into a stop button. The part of the answer already shown stays in the conversation.

Three AI providers

Gemini, OpenAI, and Anthropic are implemented behind a single Go Provider interface. The backend uses the first one that has an API key, in the order Gemini, OpenAI, then Anthropic.

Demo mode without an API key

If every API key is empty, the backend uses a mock provider that streams a sample answer word by word (about 35 ms per word), which can be stopped just like a real one.

Backend status badge

When the page loads, the frontend calls /api/health and shows the status in the header: "Gemini aktif", "Terhubung", "Mode demo", or "Backend mati".

Readable error messages

Provider API errors are translated into user-facing messages, such as Gemini quota exceeded, OpenAI credit exhausted, or API key rejected, and shown above the input box.

Suggested prompts & new chat

The start screen shows three suggested prompts that can be clicked directly. The "Chat baru" (new chat) button stops any running answer and clears the conversation.

Friendly, responsive UI

A bright design with a robot mascot, a typing indicator before the first word arrives, auto-scroll to the latest message, and an input that grows with its content. Enter sends, Shift + Enter adds a new line.

ARCHITECTURE & DATA FLOW

Browser (React)
Keeps chat history in state, sends POST /api/chat, and reads the stream with fetch + ReadableStream.

→

Vite proxy
In development, /api requests are forwarded to localhost:8080, so no CORS is needed.

→

Go ChatHandler
Validates and trims the history, then writes SSE events and flushes after every chunk.

→

Provider
Gemini, OpenAI, Anthropic, or Mock; chosen once at server start.

→

API model AI
The provider's streaming endpoint; not called in demo mode.

- The backend is stateless: every new message resends the whole history, and the backend forwards only the last 40 messages (MAX_HISTORY), making sure the conversation starts with a user message.
- The event format sent to the browser is the same for every provider: {"type":"delta"}, {"type":"done"}, or {"type":"error"}. The frontend does not need to know which provider is in use.
- Each provider reads its own API's SSE stream with bufio.Scanner, extracts the text, and calls an onDelta callback that writes the event to the browser.
- For production, the backend builds into a single binary and the frontend into static files; the backend address is set with VITE_API_URL and ALLOWED_ORIGIN.

STACK BY LAYER

Backend	Go 1.22 · net/http ServeMux · encoding/json · bufio · context
Frontend	React 18 · Vite 5 · Fetch API + ReadableStream · AbortController · Plain CSS
AI	Gemini API (streamGenerateContent) · OpenAI API (Chat Completions) · Anthropic API (Messages) · Mock provider
Testing & build	go test (httptest) · go build · vite build

TECHNICAL DECISIONS

1. **SSE instead of WebSocket**
   Data only flows one way, from server to browser, for the length of one answer. SSE is just a normal HTTP response (text/event-stream + Flush), with no extra protocol or long-lived connection to manage. The frontend reads it with fetch rather than EventSource, because EventSource only supports GET and cannot send the chat history in a body.

2. **Stop cancels all the way to the provider**
   The stop button calls AbortController.abort(), which cancels the fetch request. In Go, r.Context() is cancelled too, and since the upstream API request is built with NewRequestWithContext, it is cut off as well, so the model stops generating tokens. The handler then exits without sending an error, and the frontend ignores the AbortError so the partial answer stays visible.

3. **Provider picked from the API key**
   Instead of adding a separate "provider" setting, the backend simply checks which key is set. Gemini comes first because a free key is easy to get from Google AI Studio. Adding a provider only takes one file that implements Name() and Stream().

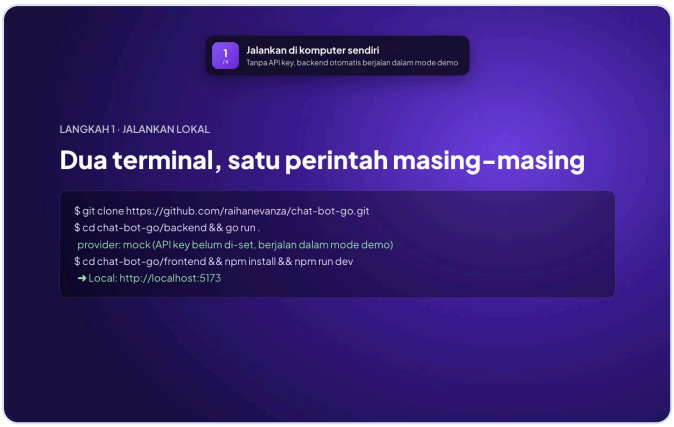
4. **No external Go dependencies**
   go.mod has no require entries at all. Routing uses the Go 1.22 ServeMux, which supports patterns like "POST /api/chat", provider APIs are called directly with net/http, and .env is read by a small hand-written parser that never overrides existing environment variables. The goal was to understand every layer and keep the build to a plain "go build".

5. **Demo mode as a fallback**
   A public repo should be usable without signing up for a paid service. The mock provider uses the same interface and respects context cancellation, so streaming, the stop button, and the status badge behave exactly as they would with a real model.

6. **Streaming that proxies do not hold back**
   The handler sends Cache-Control: no-cache and X-Accel-Buffering: no, then calls Flush after each event so nginx or other proxies do not buffer the response. The server WriteTimeout is deliberately left unset because long answers can take a while.

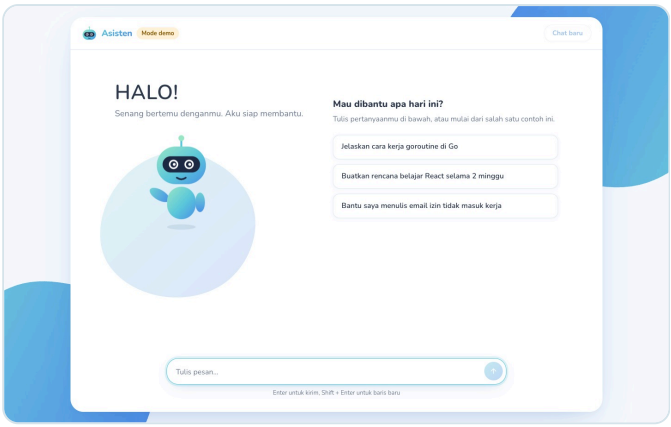
HOW TO USE IT

These steps follow the same order as the tutorial video.



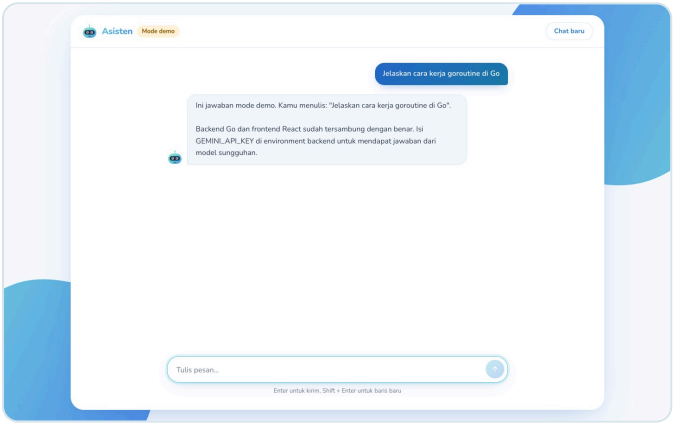
1 Clone the repo & start the backend

Clone github.com/raihanevanza/chat-bot-go, go into the backend folder, and run "go run .". Without a .env file the log shows "provider: mock" and the server runs at http://localhost:8080 in demo mode.



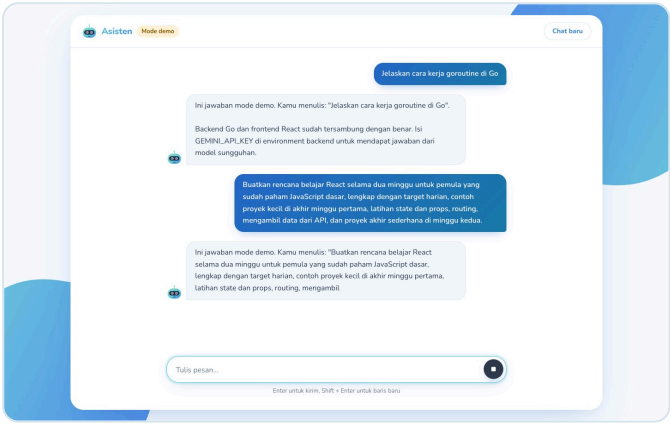
2 Start the frontend

In a second terminal, go into the frontend folder and run "npm install" then "npm run dev". Open http://localhost:5173; the header badge shows "Mode demo".



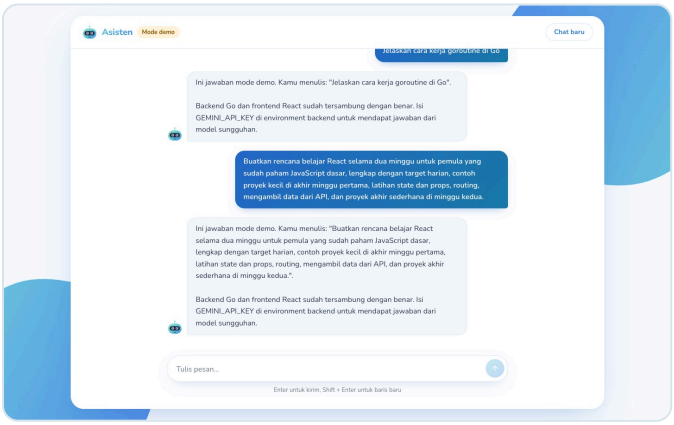
3 Send the first message

The start screen greets you with "Halo!" and shows three suggested prompts. Click one, or type in the "Tulis pesan..." box and press Enter or the arrow button.



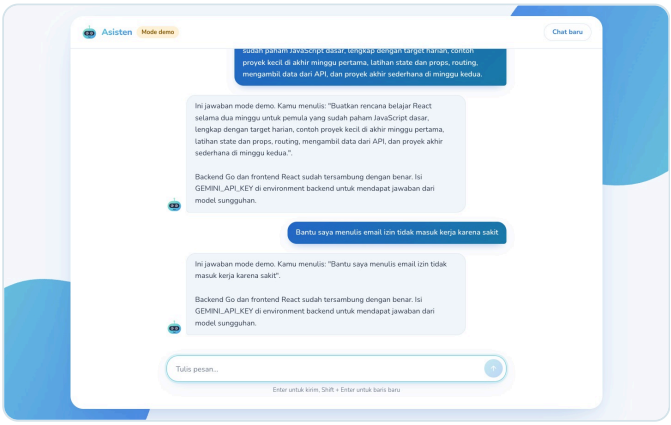
4 Watch the answer stream

A typing indicator appears briefly, then the answer appears word by word next to the mascot, and the message list scrolls along with it.



5 Stop the answer

While the answer is streaming, the send button turns into a square stop button. Click it: the answer stops immediately and the part already shown stays.



6 Continue or start a new chat

Send a follow-up message to continue the conversation (Shift + Enter for a new line). Click "Chat baru" at the top right to clear the conversation and return to the start screen.

7 **Optional: use a real model**

Copy backend/.env.example to .env, fill in GEMINI_API_KEY (or an OpenAI/Anthropic key), and restart the backend. After reloading the page, the badge changes to "Gemini aktif" or "Terhubung".

LIMITATIONS & NOTES

- There is no live demo; the app has to run locally because it calls paid APIs with my own key.
- Chat history lives only in browser memory, so it is lost on page reload. There is no login or request rate limiting yet.
- Answers are shown as plain text; Markdown from the model (headings, lists, code blocks) is not rendered yet.
- Demo mode always replies with the same template (echoing the user's message), not a real answer to the question.
- The interface text is Indonesian only, and specific error messages exist only for Gemini and OpenAI. Unit tests currently cover only the Gemini provider.

WHAT'S NEXT

- Render Markdown in answers (e.g. with react-markdown) so lists and code blocks display properly.
- Persist chat history in a database (SQLite or PostgreSQL) and add login plus rate limiting.
- Add unit tests for the OpenAI and Anthropic providers, then deploy a public demo.